WEB APPLICATION FOR REAL-TIME ASCII ART GENERATION

A MICRO PROJECT REPORT

Submitted by

Shashank Patel (2411021061117)

Rishu Gupta (2411021061138)

Pankaj Pandurang Tike (2411021061162)

Priyanshu Kumar (2411021061114)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING / INFORMATION TECHNOLOGY

Under the Supervision of

Mr. AAKASH KUMAR SHARMA



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

OCTOBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING
ALLIANCE UNIVERSITY, BENGALURU
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING / INFORMATION TECHNOLOGY

CERTIFICATE

This certifies that the Micro Project work titled "Web Application for Real Time ASCII Art Generation" has been genuinely carried out by Shashank Patel (2411021061117), Rishu Gupta (2411021061138), Pankaj Pandurang Tike (2411021061162) and Priyanshu Kumar (2411021061114) as part of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering / Information Technology for the year 2026.

Supervisor: **Mr. AAKASH KUMAR SHARMA**
Assistant Professor / Associate Professor / Professor
Dept. of Computer Science and Engineering / IT

DECLARATION

This is to state that the report entitled “Web Application for Real Time ASCII Art Generation” has been prepared as part of the Programme - B.tech in Computer Science and Engineering / Information Technology, under the supervision of Mr. Aakash Kumar Sharma. We confirm that the report is our own work carried out as part of our Micro Project. It is not a copy of any work done by anyone else. We also confirm that the contents of the report and the views expressed therein have been discussed and agreed upon with the Mentor.

NAME	REGESTRATION NO.	SIGNATURE
Shashank Patel	2411021061117	
Rishu Gupta	2411021061138	
Pankaj Pandurang Tike	2411021061162	
Priyanshu Kumar	2411021061114	

ACKNOWLEDGEMENT

The sense of satisfaction that comes with the successful completion of the job would be utterly incomplete without a nod to the people who made it all possible, whose constant guidance and encouragement have been the very foundation of all our efforts.

We are very grateful to our supervisor Mr. Aakash Kumar Sharma for his constant and inspiring guidance and co-operation throughout the course of this project. His sound advice and valuable inputs are of immense worth.

We'd like to thank Dr OSWALT MANOJ S, Dr SHEKHAR R (Head of Department) and Dr Vullikanti Vivek (Deputy Director/ASAC) for their encouragement and co-operation at various stages of the project.

We'd also like to extend our special thanks to Dr. Senbagavalli M, Associate Professor, for her invaluable guidance and support throughout the Micro Project as part of the Full Stack Development course.

We take this opportunity to express our deepest sense of gratitude and our most heartfelt thanks to the Management of Alliance University for the world class infrastructure, the very congenial atmosphere and the encouragement they have given us.

We'd like to express our very deep thanks to the teaching and non-teaching staff of the department who have stood with us throughout the project and have been so very much on our side in making it a success.

We place our highest thanks to our parents, friends and well wishers who have all been so very helpful in putting together the report on this project.

Shashank Patel (2411021061117)

Rishu Gupta (2411021061138)

Pankaj Pandurang Tike (2411021061162)
Priyanshu Kumar (2411021061114)

ABSTRACT

ASCII art is art made using plain text characters. There are loads of converters out there. The typical converter takes an image as input, produces some ASCII art and that's it. The output has no home, no place to be. We wanted to do better. Our project is a MERN web app which takes a live webcam feed, a video file or a still image and turns it into ASCII art as it happens. The result can be saved by signed in users.

The conversion happens in the browser. Each frame goes onto a small canvas, has its brightness manipulated (contrast, brightness, invert), each cell is then replaced with a character from one of five character sets and is painted in green, amber or full colour. Export is available as PNG, JPG, WebP, SVG or plain text. The server is only relevant when the user hits Save. The React client then makes a call to the Express REST API which validates the JWT, validates the body via Zod and stores the artwork in MongoDB via Mongoose.

On top of that core we've got accounts (bcrypt and JWT), a personal gallery with full CRUD, public/ private visibility, likes, search with pagination and saved presets. The six API tests that don't require a database all pass. The database backed CRUD test is there too, just can't run it as there's no MongoDB instance in our environment.

Keywords: ASCII art, MERN stack, React, Express, MongoDB, REST API, JWT, HTML5 Canvas.

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Full form / meaning
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BSON	Binary JSON (MongoDB storage format)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
ER	Entity Relationship
FSD	Full Stack Development
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
MVC	Model View Controller
npm	Node Package Manager
ODM	Object Document Mapper
REST	Representational State Transfer
SPA	Single Page Application
SVG	Scalable Vector Graphics
UI	User Interface
URI	Uniform Resource Identifier
Zod	TypeScript-first schema validation library

CONTENTS

Page No.

Title Page i

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

List of Symbols & Abbreviations vi

Contents vii

1. Introduction 1

2. Literature Review 3

3. Identification of the Problem / Scope of the Project 5

4. Proposed Idea / Solution 6

5. Objectives 7

6. Process / Methodology 8

6.1 Identify the needs of customer and users through design thinking methods

6.2 Translate the needs of customer into product / process / project

6.3 Specification of the product / process / project

6.4 Designing the solution for required customer needs

6.5 Process / Flow / Design Diagram

7. Requirements 13

7.1 Hardware Requirements

7.2 Software Requirements

7.3 Datasets (if any)

7.4 Technologies and Tools

8. Novelty of the Idea / Project 15

9. Experimentation 16

10. Results and Discussion 18

11. Conclusion and Future Directions 23

References 24

Source Code 25

Submission Page of the Paper 33

Plagiarism Report 34

1. INTRODUCTION

1.1 Background

ASCII art is of course the old trick of drawing pictures using printable characters. Converters are generally of two types. The tone based ones preserve the light and dark pattern of the original. The structure based ones attempt to follow the outlines and that generally looks better [1], at the cost of a lot more computing power as it's effectively an optimisation problem [1], [2], [3]. Tone based conversion is just a lookup from brightness to character. It's cheap and fast enough for video.

Browsers can read a camera stream and the pixels of a canvas with no plugin [7], [8]. Conversion can therefore stay on the user's device with nothing having to be uploaded. The only thing that a stand-alone converter is missing is memory (accounts, storage, sharing, saved settings etc). A full stack application can provide all of that.

1.2 About the project

Web app for real time ASCII art generation (the code refers to it as ASCII_CAM) which takes a camera feed (or video or image) and turns it into ASCII art on screen. Signed in users can save the output to a gallery in MongoDB, make it public or private, rename it, delete it and like other people's public work. Favourite settings are saved as presets. The stack is MERN: React on the front end, Node/Express for the REST API and MongoDB on the back end via Mongoose. The browser never speaks to the database at all. All communication goes via the API.

1.3 Key features

- **Live conversion.** Webcam, video file or image converted live on screen via the HTML5 Canvas API. No plugin required.
- **Settings:** width from 20 to 500 characters, contrast, brightness, invert, mirror, five character sets and three colour modes. Change one and the artwork updates instantly.
- **Export.** PNG, JPG, WebP, SVG, or plain text.
- **Gallery.** Logged in users save the art to MongoDB and can then view, rename, change its visibility or delete it. The public gallery has search, a source filter, sorting, tags and pagination so it remains a usable gallery once it's full.
- **Sharing and likes.** A public artwork gets its own page which anyone can open. Signed in users can like or unlike it.
- **Presets.** Save a preferred look once and apply it in one click.

- **Accounts.** Register and log in with a hashed password and a JWT. Google and GitHub sign in are both in the code but they only work if you've got the keys configured and we didn't actually test them.

1.4 Use cases

Use case	Who uses it	How the application helps
Create ASCII art from a photo	Any user, designers, hobbyists	Upload an image, adjust the settings and get ASCII output immediately.
Live webcam ASCII	Content creators, event audiences	The camera feed is rendered as moving ASCII art in real time.
Convert a video clip	Creators, students	Upload a video and view it as ASCII frames.
Save and manage a gallery	Registered users	Save the results to MongoDB and then you can view, edit or delete them at a later stage.
Share artwork publicly	Users sharing with friends	Mark an artwork as public and share the link so that others can view it without having to log in.
Reuse favourite settings	Frequent users	Save a preset and apply the same look to new images or videos quickly.
Learning and demonstration	Students and teachers	Shows how images map to characters and a complete React, Express and MongoDB flow.

1.5 Team and roles

We split the work by layer. Each of us owns one part of the stack, and each is expected to understand the whole flow, not only their own slice.

Team member	Role	Core ownership
Pankaj Pandurang Tike	React Frontend Development	UI, components, routing, state management, API integration on the client, ASCII conversion engine
Shashank Patel	Express / Node.js Backend and APIs	Server setup, REST endpoints, controllers, middleware, validation, error handling
Rishu Gupta	MongoDB and Database	Schema design, Mongoose models, CRUD queries, indexes, data relationships
Priyanshu Kumar	Authentication, Integration, Testing and Deployment	JWT authentication, front end and back end integration, API testing, bug fixing, hosting

1.6 Organisation of the report

Chapter 2 covers related work. Chapters 3 to 5 give the problem, the idea and the objectives. Methodology and design are in chapter 6, requirements in 7, novelty in 8, experiments and results in 9 and 10, and chapter 11 ends with future work. The key source code follows the references.

2. LITERATURE REVIEW

Paper 1: Structure-based ASCII Art

Abstract: Xu, Zhang and Wong, ACM Transactions on Graphics, 2010 [1]. Tone-based methods, the authors argue, give a halftone look and need a lot of text resolution to look good. Their answer is to match the main line structure of the image against character shapes, using a similarity measure that tolerates small misalignment, then deforming the image slightly to fit. It is posed as an optimisation.

Accomplishments and Limitations: The results look hand-made. It is still an offline optimisation, built for fixed-width characters and line-like content. As far as we can tell it is not meant for frames that change thirty times a second.

Scope: Gives us the vocabulary of tone-based and structure-based art. We deliberately chose the tone-based route because it can run on every camera frame in the browser. Structure-based conversion goes on the future-work list.

Paper 2: ASCII Art Synthesis from Natural Photographs

Abstract: Xu, Zhong, Xie, Liu, Qin and Wong, IEEE Transactions on Visualization and Computer Graphics, vol. 23, no. 8, 2017 [2]. Photographs are harder than drawings. Texture swamps the weak structure a viewer actually cares about. The paper models structure separately from texture (a non-classical receptive field model, with adjustable texture suppression) and then places proportional-font characters by optimisation.

Accomplishments and Limitations: It copes with a wide range of photographs and lets the amount of structure follow the text resolution. The price is a heavy pipeline aimed at single images, and the work is a research implementation, not an application with accounts or storage.

Scope: Shows how far photo conversion can be pushed, and why text resolution matters. We expose resolution as a user setting and keep the pipeline light enough for live use.

Paper 3: Texture-Aware ASCII Art Synthesis with Proportional Fonts

Abstract: Xu, Zhong, Xie, Qin, Chen, Jin, Wong and Han, Expressive 2015, Eurographics Association [3]. Another structure-based method, this time for photographs or drawings. It keeps both contours and

fine texture by using a multi-orientation phase congruency feature, compares shapes with a deformation-tolerant metric, and picks characters and their positions together with dynamic programming.

Accomplishments and Limitations: Proportional fonts are supported, and the authors report better visual quality than earlier methods. It is a stand-alone algorithm, with no web interface, no persistence and no sharing.

Scope: Confirms that careful character choice makes richer output. Our app sticks to a fixed-width grid (6 by 12 pixels) so the same text can be stored, redrawn and exported as TXT or SVG without surprises.

Any other Sources (Web Site, Magazine, News Paper, Articles, News Channels, Blogs, Books)

The engineering side also leans on these non-academic sources:

- MDN Web Docs for the Canvas API and getUserMedia [7], [8]. Pixel access, drawing text on a canvas, capturing a camera stream.
- The official docs of React, Express, MongoDB, Mongoose and Zod [9]–[13], which cover hooks, the middleware chain, indexes and schema validation.
- RFC 7519 on JSON Web Tokens [4] and the bcrypt paper [6]. Stateless login, and slow password hashing on purpose.
- Fielding's dissertation [5], where REST comes from, and the style the API follows.

Online and command-line image-to-ASCII tools are everywhere. Most convert one image and return text. We did not find one (and we did not search exhaustively) that also offers live camera input, accounts, public sharing and presets in a single place. That is the gap this project aims at.

3. IDENTIFICATION OF THE PROBLEM / SCOPE OF THE PROJECT

3.1 Problem statement

Say you want ASCII art from a photo, a video clip and your webcam. Today that usually means three tools. Nothing gets saved, so coming back tomorrow means starting again. Character set, width and contrast have to be set up every time, and sharing is done by passing files around. And a student who just wants to see how an image turns into characters has no small example that also shows React, Express and MongoDB working together.

3.2 Specific problems identified

- No single tool takes camera, video and image input with live adjustment.
- Nothing is saved. Come back tomorrow and the art is gone.
- Favourite settings cannot be stored and re-applied.
- Sharing is manual: export a file, send the file.
- No login, so no ownership, and no way to separate private work from public.

3.3 Scope of the project

In scope:

- Tone-based ASCII conversion of webcam, video and image sources in the browser.
- User registration and login with JWT; a personal gallery with full CRUD; public or private visibility; likes.
- Conversion presets, search, filter, sort and pagination of the gallery, and export to PNG, JPG, WebP, SVG and TXT.
- A REST API with validation, authentication and central error handling, backed by MongoDB.

Out of scope (future work):

- Structure-based ASCII art and machine-learning based conversion.
- Server-side video processing, animated GIF or video export of the ASCII output.
- Comments, an admin moderation console and production deployment.

4. PROPOSED IDEA / SOLUTION

The idea is simple. Do the heavy, time-sensitive part (converting each frame) in the browser. Use the server only for things that must be shared or remembered, meaning accounts, the gallery and presets.

4.1 Core idea

- **Convert locally.** The React client samples each frame onto a small canvas, maps brightness to characters and draws the text. Nothing is uploaded. The preview stays fast, and camera data never leaves the device.
- **Persist on purpose.** Only when the user presses Save does the client send the ASCII text, a small thumbnail, the settings and the source type to the API.
- **Keep layers apart.** React for presentation and state. Express for validation, authentication and business logic. MongoDB for users, artworks, presets and FAQs.
- **Protect ownership.** Every update and delete query carries the owner's id. One user cannot touch another user's art, even by guessing an id.

4.2 Solution architecture

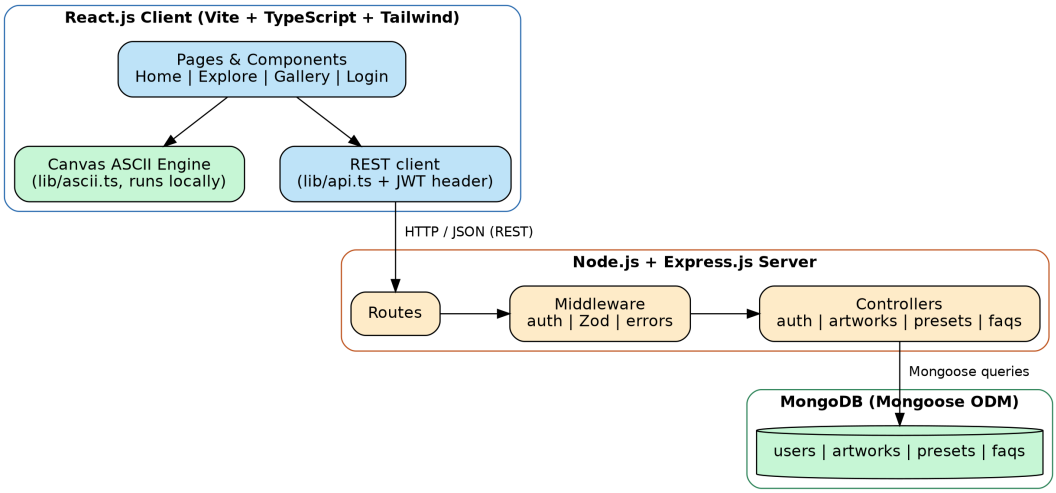


Figure 4.1: Three-tier architecture of the application

The browser talks to Express over HTTP and JSON, nothing else. A request passes the middleware (JWT check, Zod validation), reaches a controller, and the controller reads or writes MongoDB through a Mongoose model. The ASCII engine sits in the client and never calls the server.

5. OBJECTIVES

What we set out to do:

- Build a working full stack MERN application, not just a front end, where every screen reads and writes real data.
- Convert camera, video and image input to ASCII art in real time, in the browser, with the Canvas API.
- Give users control over the look: character set, width, contrast, brightness, invert, mirror, colour mode.
- Store users, artworks and presets in MongoDB with sensible Mongoose schemas, indexes and relationships.
- Provide REST APIs with full CRUD, validation, correct status codes and one central error handler.
- Secure it: bcrypt hashing, JWT authentication, owner-only updates, a rate limit on login.
- Allow public sharing and reusable presets.
- Test the API automatically, and write down the architecture, database design and API so the team can follow it.

6. PROCESS / METHODOLOGY

1. Identify the needs of customer and users through design thinking methods.

We framed the needs with the five design-thinking stages, using the user groups and use cases from the project brief (hobbyists and designers, content creators, students and teachers). We did not run a formal survey.

Stage	What was done	Outcome
Empathise	Considered how hobbyists, creators and students would use an ASCII converter.	Users want instant visual feedback, control over the look, and a way to keep and share results.
Define	Summarised the pain points (Chapter 3).	Need one tool for camera, video and image with saving, sharing and presets.
Ideate	Compared server-side and browser-side conversion.	Convert in the browser; use the server only for accounts and storage.
Prototype	Built the converter first, then the API and database, then the gallery.	A working vertical slice: convert, save, list, edit, delete.
Test	Wrote API tests and CRUD scenarios.	Validation, authentication and ownership rules checked automatically.

2. Translate the needs of customer into product / process / project.

Each need became a feature, an API or database element, and an owner in the team.

User need	Feature in the project	Where it lives
See art instantly from camera, video or image	Canvas ASCII engine and dashboard	client: lib/ascii.ts, AsciiDashboard.tsx
Control the look of the art	Character set, width, contrast, brightness, invert, mirror, colour mode	client: AsciiDashboard.tsx
Keep my art	Save to gallery; list, rename, delete	server: routes/controllers artworks; MongoDB artworks
Share art with a link	Public / private flag and public artwork page	artworks.isPublic; ArtworkPage.tsx
Reuse my favourite look	Presets (create, list, replace, delete)	server: presets; MongoDB presets
Keep my data private	Register, login, JWT, owner checks	server: auth, middleware/auth.js; users collection

3. Specification of the product / process / project

Functional requirements

- FR1: Convert webcam, video and image input to ASCII art on screen.
- FR2: Allow the user to change the conversion settings while the art is being shown.
- FR3: Register and log in; the password is stored only as a bcrypt hash.
- FR4: Save an artwork (title, text, thumbnail, settings, source, visibility, tags) for the logged-in user.
- FR5: List public art and the user's own art with search, filter, sort and pagination; read a single artwork.
- FR6: Update the title, visibility and tags of own art; delete own art.
- FR7: Like or unlike a public artwork once per user.
- FR8: Create, list, replace and delete personal presets; preset names are unique per user.

Non-functional requirements

- Performance: conversion cost per frame is proportional to columns × rows; the full-size art text is excluded from list queries.
- Security: hashed passwords, JWT with expiry, Helmet headers, CORS allow-list, request size limit and login rate limit.
- Reliability: input validation on client and server, one error format, no internal details leaked on server errors.
- Maintainability: separate client and server folders, routes → controllers → models structure.

Conversion settings and limits

Setting	Allowed values	Meaning
cols	20 – 500	Width of the art in characters
contrast	0.3 – 3	Multiplier around mid-grey
brightness	−0.5 – 0.5	Offset added to luminance
charset	detailed, classic, blocks, binary, matrix	Characters ordered from dark to light
mode	green, amber, color	Colour used to paint the text
invert / mirror	true or false	Reverse brightness / flip horizontally

REST API specification (base URL /api, JSON bodies, Authorization: Bearer <token>)

Method	Endpoint	Auth	Purpose	Success	Errors
GET	/health	–	API and database status	200	–

Method	Endpoint	Auth	Purpose	Success	Errors
POST	/auth/register	–	Create account	201	400, 409
POST	/auth/login	–	Log in, return JWT	200	400, 401
GET	/auth/me	Yes	Current user	200	401
GET	/artworks	Public / Yes	List art (scope, search, sort, page)	200	400, 401
GET	/artworks/:id	Optional	Read one artwork with text	200	400, 404
POST	/artworks	Yes	Create artwork	201	400, 401, 413
PATCH	/artworks/:id	Owner	Update title / visibility / tags	200	400, 401, 404
DELETE	/artworks/:id	Owner	Delete artwork	204	401, 404
POST	/artworks/:id/like	Yes	Like or unlike	200	401, 404
GET	/presets	Yes	List my presets	200	401
POST	/presets	Yes	Create preset	201	400, 409
PUT	/presets/:id	Yes	Replace preset	200	400, 404
DELETE	/presets/:id	Yes	Delete preset	204	404
GET	/faqs	–	FAQ list from MongoDB	200	–

4. Designing the solution for required customer needs.

The design follows the three tiers of Figure 4.1. Below: data, request path, errors.

Database design. MongoDB holds four collections. One user owns many artworks and many presets, linked by the user's ObjectId, and populate pulls in the author's name for the public gallery. The ASCII text lives inside the artwork document. Usually that is a few KB. At the maximum width it reaches roughly 100 KB, still far below MongoDB's 16 MB document limit, and list queries skip the text field anyway.

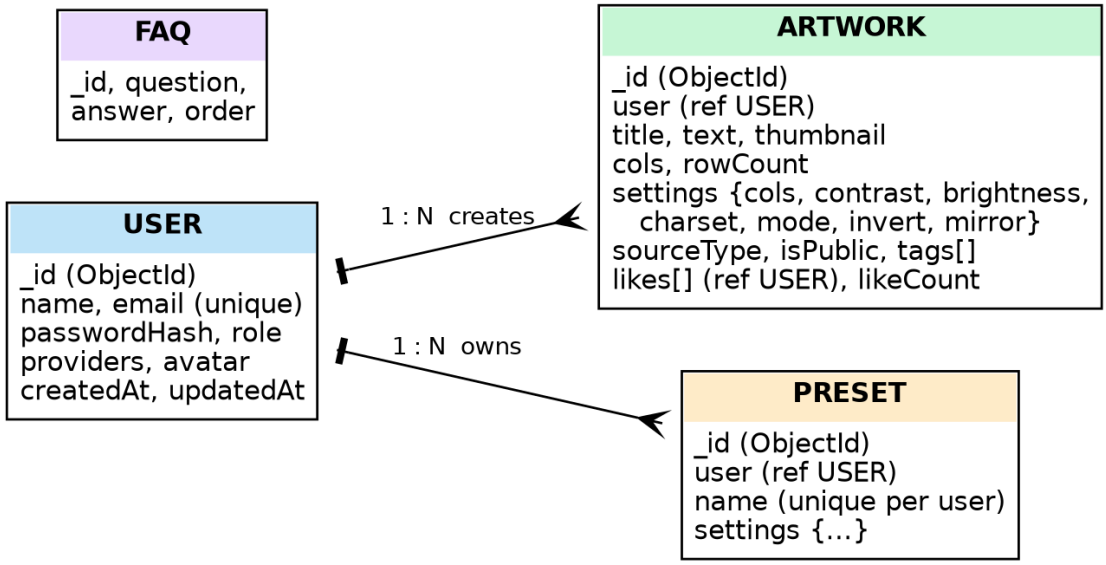


Figure 6.1: Database schema (entity relationship) diagram

Collection	Key fields	Indexes and rules
users	name, email, passwordHash (bcrypt, never returned), role, providers	email unique and lowercase
artworks	user (ref), title, text, thumbnail, cols, rowCount, settings, sourceType, isPublic, tags, likes, likeCount	{user, createdAt}; {isPublic, createdAt}; {isPublic, likeCount, createdAt}; {isPublic, tags}
presets	user (ref), name, settings	unique {user, name}
faqs	question, answer, order	seeded on first start

Request path. Figure 6.2 follows one save. Every other endpoint repeats the same pattern: router, auth, validation, controller, model.

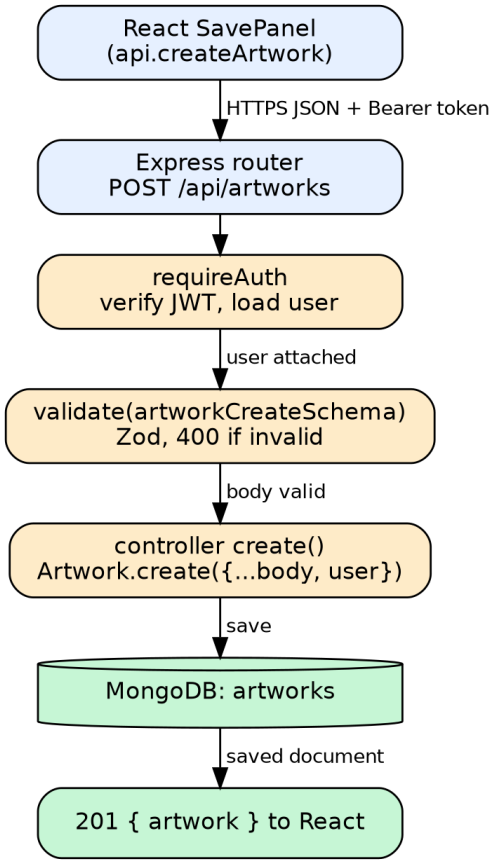


Figure 6.2: Request path for POST /api/artworks

Error handling. Controllers are wrapped in an asyncHandler, so any thrown error lands in one errorHandler that answers in a single JSON format.

Condition	Status	Message
Zod validation failure, bad ObjectId, malformed JSON	400	Validation failed / Invalid id / Malformed JSON
Missing, invalid or expired token	401	Authentication required / Invalid or expired token
Artwork not found or not visible to the caller	404	Artwork not found
Duplicate email or preset name	409	That value already exists

Condition	Status	Message
Request body too large	413	Payload too large
Unexpected failure	500	Internal server error (details logged only)

6.5 Process / Flow / Design Diagram

Figure 6.3 traces the process from picking a source to managing saved art. Everything up to the login check runs in the browser. After it, the REST API takes over.

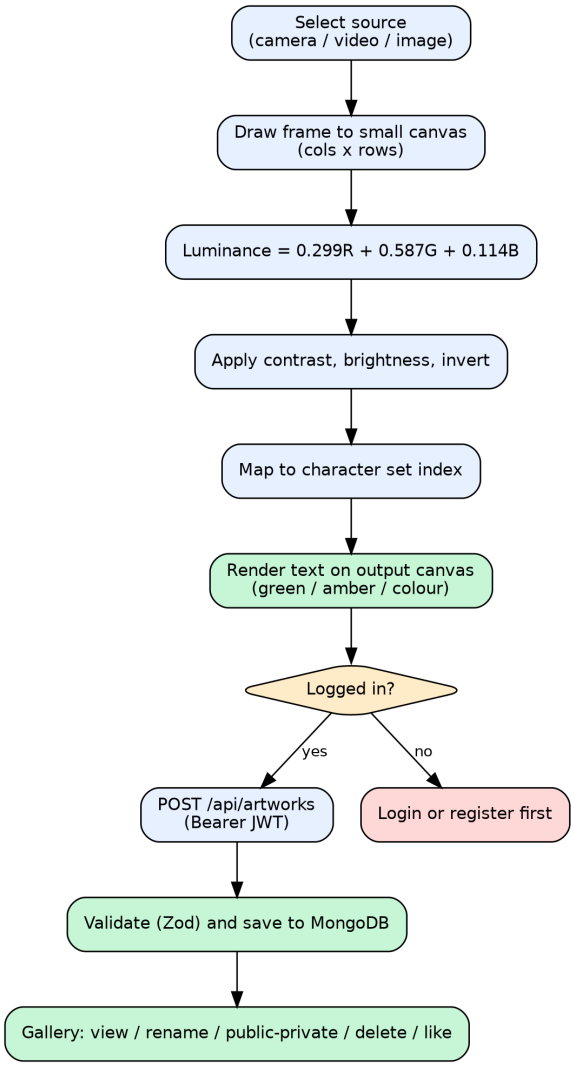


Figure 6.3: Process flow of the application

Conversion algorithm. For every frame the engine (i) draws the source on a canvas of cols × rows pixels, where rows = round(cols × height × 6 / (width × 12)) so that the 6 × 12 pixel character cell keeps the picture's aspect ratio; (ii) computes luminance L = 0.299R + 0.587G + 0.114B; (iii) applies contrast and brightness as $L' = (L - 0.5) \times \text{contrast} + 0.5 + \text{brightness}$ and clamps it to 0..1; (iv) inverts it if requested; and (v) selects the character at index round($L' \times (n - 1)$) of the chosen character set and paints it.

7. REQUIREMENTS

a. Hardware Requirements

Item	Requirement
Development computer	Laptop or desktop, 4 GB RAM minimum (8 GB recommended), dual-core processor or better, about 1 GB free disk space
Camera	Built-in or USB webcam (needed only for the live camera mode)
Network	Internet access to install packages and to use a MongoDB Atlas cluster (not needed with local MongoDB)
Client device	Any modern computer or phone with a current browser that supports canvas and camera access

b. Software Requirements

Item	Requirement
Operating system	Windows, macOS or Linux
Runtime	Node.js 18 or later with npm
Database	MongoDB (local, Docker via docker-compose, or MongoDB Atlas)
Browser	Current Chrome, Edge, Firefox or Safari
Editor and tools	Visual Studio Code, Git, Postman or Thunder Client, MongoDB Compass

c. Datasets (If any)

There is no external dataset. The input is whatever camera feed, video or image the user supplies. For demos, the seed scripts help: default FAQ entries are created on first start, and npm --prefix server run seed:gallery adds a few sample public artworks.

d. Technologies and Tools

Layer	Technology	Used for
Frontend	React 18, TypeScript, Vite, Tailwind CSS, React Router 6, lucide-react	UI, routing, state, forms, calling the API
Conversion	HTML5 Canvas API, MediaDevices getUserMedia, requestAnimationFrame	Capture and render ASCII frames
Backend	Node.js, Express 4	REST API, routing, middleware
Database	MongoDB, Mongoose 8	Persistent storage and modelling
Authentication	jsonwebtoken, bcryptjs	Stateless login and password hashing
Validation	Zod	Request body and query validation
Security	Helmet, CORS, express-rate-limit	Headers, origin allow-list, brute-force limit
Testing	Node.js test runner, Supertest	Automated API tests
Tools	Git, VS Code, Postman, MongoDB Compass, Docker	Version control, editing, API and database checks

8. NOVELTY OF THE IDEA / PROJECT

Converters exist. Galleries exist. So do full stack demos. What we think is new is putting them together, plus a few design choices:

- **Convert in the browser, persist on the server.** The live preview never leaves the device. The server is contacted for accounts, saving and sharing, nothing else.
- **Three sources, one engine.** Camera, video and image share the same conversion code and the same settings panel.
- **The art is stored as text.** We keep the actual ASCII, a small thumbnail and the exact settings. So it can be shown again, re-exported as TXT, SVG or image, or recreated.
- **Presets are stored per user.** Each one has a name, and the name must be unique for that user.
- **Public versus private.** A visibility flag separates them. Private art goes back only to its owner, and update and delete queries include the owner's id.
- **Likes that cannot double count.** A like is one atomic update guarded by the filter, so a double click does nothing extra. The count lives in the document, which makes sorting by top a plain index lookup.
- **Export options.** PNG, JPG, WebP, SVG, TXT.

9. EXPERIMENTATION

9.1 Experimental setup

All experiments use the project source as it stands. We applied the conversion rules from lib/ascii.ts (luminance formula, contrast and brightness, character mapping, 6 × 12 cell ratio) to a synthetic 200 × 200 picture, a shaded sphere on a gradient, because the effect of each setting is easy to see on it. The API tests ran with the Node.js test runner (Node v22), using npm test in the server folder. No MongoDB instance was available, so the database test was skipped.

9.2 Experiment 1: effect of the character set

Same image, 44 columns, contrast 1.2, brightness 0, three character sets: classic (10 levels), blocks (5) and detailed (70). Output is in Chapter 10 (Figure 10.1).

9.3 Experiment 2: effect of width on storage

Stored size depends on the column count. For a 4:3 source the number of rows is round(cols × 0.375), and the text is (cols + 1) × rows − 1 characters, because rows are joined with newlines. Table 9.1 has the numbers.

Columns	Rows (4:3 source)	Characters stored	Approx. size
20	8	167	0.2 KB
60	22	1,341	1.3 KB
120	45	5,444	5.3 KB
250	94	23,593	23.0 KB
500	188	94,187	92.0 KB

Table 9.1: Stored text size against width (single-byte characters)

9.4 Experiment 3: automated API tests

tests/api.test.js needs no database. It covers the health check, unknown routes, validation failures, malformed JSON, protected routes and bad tokens. tests/crud.test.js goes further (register, login, artwork CRUD, visibility, presets) but only runs when MONGO_URI_TEST is set. Outcomes are in Chapter 10.

9.5 Experiment 4: end-to-end scenarios

Table 9.2 lists the scenarios that define a full check from the browser or Postman, and says how each is covered.

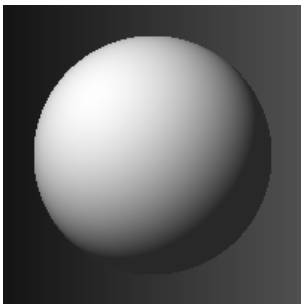
No.	Scenario	Expected result	Covered by
1	Register with valid data; repeat with the same e-mail	201 with token and user; second attempt 409	crud.test.js
2	Register with invalid e-mail or short password	400 with field details	api.test.js
3	Log in with wrong password	401 Incorrect email or password	crud.test.js
4	POST /api/artworks without token	401 Authentication required	api.test.js
5	Create an artwork, list it under “mine”, open it	201, then it appears in the list, text returned by GET :id	crud.test.js, browser
6	Rename and make public; open as a logged-out user	200; public page visible without login	crud.test.js, browser
7	Another user tries to delete the artwork	404 (owner filter), art unchanged	crud.test.js
8	Delete own artwork	204; no longer listed	crud.test.js, browser
9	Create preset; create same name again	201; second attempt 409	crud.test.js
10	Live camera, video and image conversion with settings changed	Art updates on screen; export works	Manual (browser)

Table 9.2: End-to-end test scenarios

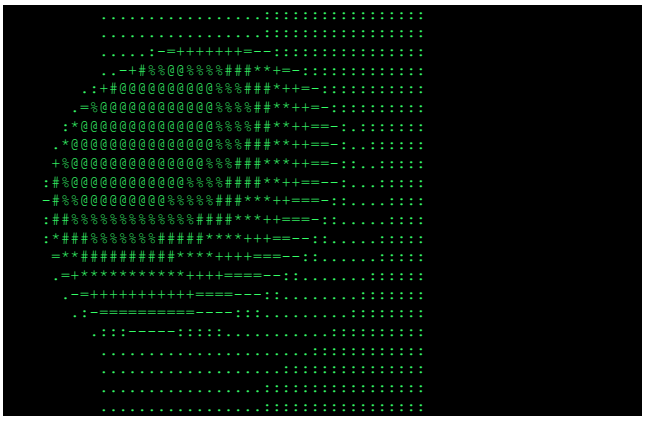
10. RESULTS AND DISCUSSION

10.1 Conversion results

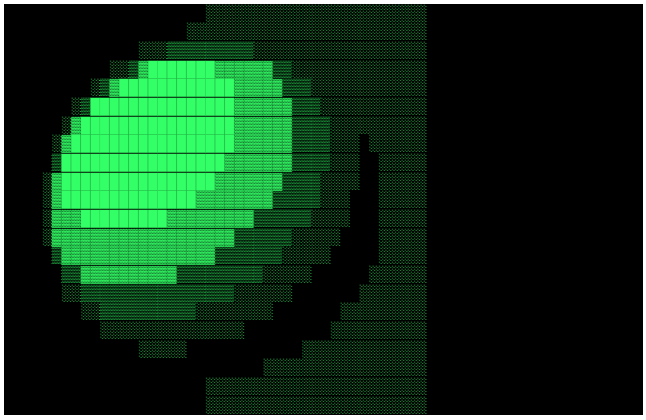
Figure 10.1 shows the test image and its conversion with three character sets, using the application's own mapping rules.



(a) Input image (200 × 200)



(b) classic, 44 columns



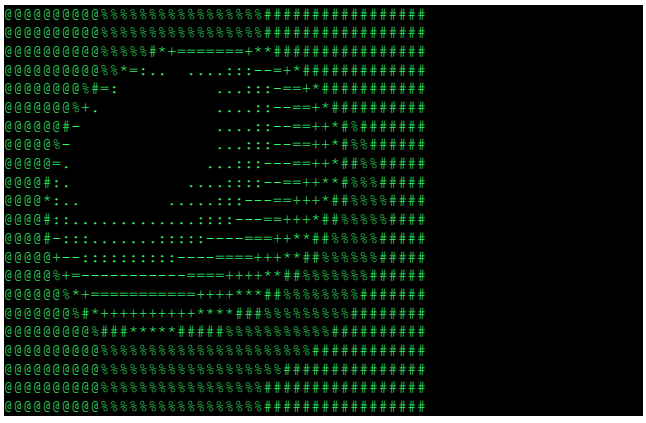
(c) blocks, 44 columns



(d) detailed, 44 columns

Figure 10.1: Effect of the character set on the same image

Figure 10.2 shows the invert option and a much lower width.



(a) classic, inverted



(b) classic, 20 columns

Figure 10.2: Effect of invert and of resolution

10.2 API test results

No.	Test	Result
1	GET /api/health returns ok	Pass
2	unknown route returns a JSON 404	Pass
3	register rejects a short password and bad email with 400	Pass
4	malformed JSON returns 400	Pass
5	protected routes require a token	Pass
6	a bad token is rejected with 401	Pass
7	auth + artwork + preset CRUD flow (crud.test.js)	Skipped (needs MONGO_URI_TEST)

Table 10.1: Result of npm test (6 passed, 0 failed, 1 skipped)

10.3 Discussion

- **Character set.** Classic has only 10 levels and still reads cleanly. Detailed (70 levels) handles smooth shading better, and the gain grows with width. Blocks give a bold, poster look. Binary and matrix are mostly for fun.
- **Resolution.** More columns, more detail, more work per frame, more bytes stored (Table 9.1). Even at 500 columns the text stays near 100 KB.
- **Invert and contrast.** Invert is needed when the art sits on a light background. Contrast and brightness can rescue a dark or washed-out source, up to a point. Past that, there is not enough information left to recover.
- **Architecture.** Converting in the browser means the live preview does not depend on network speed. Leaving the text out of list queries keeps gallery pages light. In the tests we could run, the owner-checked queries and the central error handler behaved the same way every time.
- **Limitations.** The conversion is tone-based, so it ignores edges. We did not benchmark frame rate on different devices, which is a real gap. Live camera mode needs the browser's permission, and some users will say no.

10.4 Implementation results

This space is for the results of running the application. Table 10.2 is to be filled in after the demo run, and the boxes below are for screenshots.

Feature	What was tried	Observed result	Remarks
Live camera conversion	Allow camera, change settings		
Video conversion	Upload a short clip		

Feature	What was tried	Observed result	Remarks
Image conversion	Upload a photo		
Register and login	New account, then login		
Save to gallery	Press Save while logged in		
Edit and delete	Rename, delete an artwork		
Public sharing	Make public, open link logged out		
Likes	Like and unlike		
Presets	Save and apply a preset		
Export	PNG, JPG, WebP, SVG, TXT		

Table 10.2: Implementation results (to be filled in)



Figure 10.3: Landing page



Figure 10.4: Register and login, including a validation error

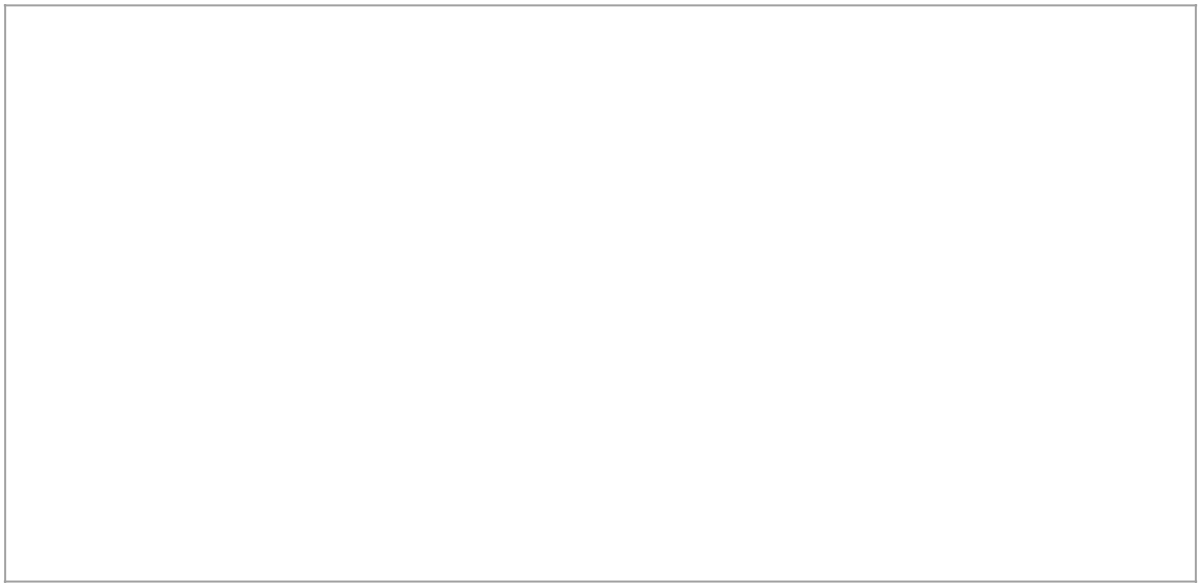


Figure 10.5: Explore page converting a live camera feed or an image

11. CONCLUSION AND FUTURE DIRECTIONS

11.1 Conclusion

The project works end to end. Open the site, point a camera or pick a file, adjust the art, log in, save it. Later it is in the gallery, where it can be edited, deleted, shared or liked. React, Express and MongoDB stay separate and meet only at a documented REST contract. Validation, JWT authentication, owner-only queries and a central error handler keep the API consistent. The tests that run without a database all passed. Each of us owned one layer. Putting the layers together was where most of our debugging time went.

11.2 Future directions

- Try structure-based or learning-based conversion for cleaner line art, building on [1]–[3]. Probably the biggest quality jump.
- Export the live output as an animated GIF or a video, and benchmark frame rate on real devices, which we never did.
- Comments, reporting and an admin console to moderate the public gallery.
- Deploy: static host for the front end, a Node host for the API, MongoDB Atlas. Add CI that runs the tests.
- More tests, including browser end-to-end ones, and run the CRUD tests against a real MongoDB in CI.
- Accessibility. A high-contrast mode, and screen reader text for gallery items.

REFERENCES

[1] X. Xu, L. Zhang, and T.-T. Wong, “Structure-based ASCII art,” ACM Trans. Graph., vol. 29, no. 4, Art. no. 52, Jul. 2010, doi: 10.1145/1778765.1778789.

[2] X. Xu, L. Zhong, M. Xie, X. Liu, J. Qin, and T.-T. Wong, “ASCII art synthesis from natural photographs,” IEEE Trans. Vis. Comput. Graphics, vol. 23, no. 8, pp. 1910–1923, Aug. 2017.

[3] X. Xu, L. Zhong, M. Xie, J. Qin, Y. Chen, Q. Jin, T.-T. Wong, and G. Han, “Texture-aware ASCII art synthesis with proportional fonts,” in Proc. Expressive 2015, Eurographics Association, 2015.

[4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[5] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[6] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annu. Tech. Conf. (FREENIX Track), Monterey, CA, USA, 1999, pp. 81–91.

[7] Mozilla, “Canvas API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[8] Mozilla, “MediaDevices: getUserMedia() method,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[9] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev

[10] OpenJS Foundation, “Express – Node.js web application framework.” [Online]. Available: https://expressjs.com

[11] MongoDB, Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/manual/

[12] “Mongoose documentation.” [Online]. Available: https://mongoosejs.com/docs/

[13] “Zod: TypeScript-first schema validation.” [Online]. Available: https://zod.dev

SOURCE CODE

Key files are listed below. The full client and server folders are submitted with the report.

Listing 1: client/src/lib/ascii.ts (ASCII conversion engine)

```
export const CHARSETS = {
  detailed: " .'`^\",:;Il!i><~+_-?][}{1)(|\\/tfjrxnuvczXYUJCLQ0OZmwqpdbkhao*#MW&8%B@$",
  classic: " .:-=+*#%@",
  blocks: " ░▒▓█",
  binary: " 01",
  matrix: " ｦｱｲｳｴｵｶｷｺｻ012345789",
} as const;
export type CharsetKey = keyof typeof CHARSETS;
export type Mode = "green" | "amber" | "color";

export interface Settings {
  cols: number;
  contrast: number;
  brightness: number;
  charset: CharsetKey;
  mode: Mode;
  invert: boolean;
  mirror: boolean;
}

export interface Frame {
  rows: string[];
  data: Uint8ClampedArray;
  cols: number;
  rowCount: number;
  mode: Mode;
}

export type Source = HTMLVideoElement | HTMLImageElement;
export const CW = 6; // cell width (px)
export const CH = 12; // cell height (px)

export function sourceSize(src: Source): [number, number] {
  return src instanceof HTMLVideoElement ? [src.videoWidth, src.videoHeight] : [src.naturalWidth,
src.naturalHeight];
}

/** Sample the source into a small canvas, map brightness to characters and paint them on `out`. */
export function renderAscii(src: Source, cols: number, s: Settings, off: HTMLCanvasElement, out: HTMLCanvasElement):
Frame | null {
  const [w, h] = sourceSize(src);
  if (!w || !h) return null;
  const rowCount = Math.max(1, Math.round((cols * h * CW) / (w * CH)));
  off.width = cols;
  off.height = rowCount;
  const octx = off.getContext("2d", { willReadFrequently: true })!;
  octx.save();
  if (s.mirror) { octx.translate(cols, 0); octx.scale(-1, 1); }
  octx.drawImage(src, 0, 0, cols, rowCount);
  octx.restore();
  const d = octx.getImageData(0, 0, cols, rowCount).data;

  const set: string = CHARSETS[s.charset];
  const n = set.length - 1;
  out.width = cols * CW;
  out.height = rowCount * CH;
  const ctx = out.getContext("2d")!;
  ctx.fillStyle = "#000";
  ctx.fillRect(0, 0, out.width, out.height);
  ctx.font = `${CH - 1}px ui-monospace, Menlo, Consolas, monospace`;
  ctx.textBaseline = "top";
  ctx.fillStyle = s.mode === "amber" ? "#ffb000" : "#33ff66";

  const perChar = s.mode === "color" || s.charset === "matrix";
  const rows: string[] = [];
  for (let y = 0; y < rowCount; y++) {
    let row = "";
    for (let x = 0; x < cols; x++) {
      const i = (y * cols + x) * 4;
      let l = (0.299 * d[i] + 0.587 * d[i + 1] + 0.114 * d[i + 2]) / 255;
      l = (l - 0.5) * s.contrast + 0.5 + s.brightness;
      l = l < 0 ? 0 : l > 1 ? 1 : l;
      if (s.invert) l = 1 - l;
      const ch = set[Math.round(l * n)];
      row += ch;
      if (perChar && ch !== " ") {
        if (s.mode === "color") ctx.fillStyle = `rgb(${d[i]},${d[i + 1]},${d[i + 2]})`;
        ctx.fillText(ch, x * CW, y * CH);
      }
    }
    if (!perChar) ctx.fillText(row, 0, y * CH);
```

```
    rows.push(row);
  }
  return { rows, data: d, cols, rowCount, mode: s.mode };
}
```

Listing 2: client/src/lib/api.ts (REST client with token header)

```
import type { Settings } from "@/lib/ascii";

const BASE = import.meta.env.VITE_API_URL || "/api";
const KEY = "ascii_cam_token";

export const getToken = () => localStorage.getItem(KEY);
export const setToken = (t: string | null) => (t ? localStorage.setItem(KEY, t) : localStorage.removeItem(KEY));

export interface User { _id: string; name: string; email: string; role: "user" | "admin"; avatar?: string;
providers?: string[] }
export interface Artwork {
  _id: string; title: string; text?: string; thumbnail?: string; cols: number; rowCount: number;
  settings: Settings; sourceType: "camera" | "video" | "image"; isPublic: boolean; tags: string[];
  user: string | { _id: string; name: string }; createdAt: string;
  likeCount: number; likedByMe: boolean;
}
export interface Preset { _id: string; name: string; settings: Settings }
export interface Faq { _id: string; question: string; answer: string }
export interface Page<T> { items: T[]; page: number; pages: number; total: number }
export interface ArtworkQuery {
  scope: "public" | "mine" | "liked"; page: number; sort?: "new" | "top"; search?: string; source?:
Artwork["sourceType"]; tag?: string;
}
export type NewArtwork = Pick<Artwork, "title" | "text" | "thumbnail" | "cols" | "rowCount" | "settings" |
"sourceType" | "isPublic" | "tags">;

export class ApiError extends Error {
  constructor(public status: number, message: string, public details?: { field: string; message: string }[]) {
    super(message);
  }
}

async function request<T>(path: string, method = "GET", body?: unknown): Promise<T> {
  const headers: Record<string, string> = {};
  if (body !== undefined) headers["Content-Type"] = "application/json";
  const token = getToken();
  if (token) headers.Authorization = `Bearer ${token}`;

  let res: Response;
  try {
    res = await fetch(BASE + path, { method, headers, body: body === undefined ? undefined :
JSON.stringify(body) });
  } catch {
    throw new ApiError(0, "Cannot reach the server. Is the backend running?");
  }
  if (res.status === 204) return undefined as T;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new ApiError(res.status, data?.error?.message || "Request failed", data?.error?.details);
  return data as T;
}

export const api = {
  register: (b: { name: string; email: string; password: string }) => request<{ token: string; user: User }>("/auth/
register", "POST", b),
  login: (b: { email: string; password: string }) => request<{ token: string; user: User }>("/auth/login", "POST",
b),
  providers: () => request<{ google: boolean; github: boolean }>("/auth/providers"),
  me: () => request<{ user: User }>("/auth/me").then((r) => r.user),

  listArtworks: (q: ArtworkQuery) => {
    const p = new URLSearchParams({ scope: q.scope, page: String(q.page), limit: "12", sort: q.sort ?? "new" });
    if (q.search) p.set("search", q.search);
    if (q.source) p.set("source", q.source);
    if (q.tag) p.set("tag", q.tag);
    return request<Page<Artwork>>(`/artworks?${p}`);
  },
  toggleLike: (id: string) => request<{ liked: boolean; likeCount: number }>(`/artworks/${id}/like`, "POST"),
  getArtwork: (id: string) => request<{ artwork: Artwork }>(`/artworks/${id}`).then((r) => r.artwork),
  createArtwork: (b: NewArtwork) => request<{ artwork: Artwork }>("/artworks", "POST", b).then((r) => r.artwork),
  updateArtwork: (id: string, b: Partial<Pick<Artwork, "title" | "isPublic" | "tags">>) => request<{ artwork:
Artwork }>(`/artworks/${id}`, "PATCH", b).then((r) => r.artwork),
  deleteArtwork: (id: string) => request<void>(`/artworks/${id}`, "DELETE"),

  listPresets: () => request<{ items: Preset[] }>("/presets").then((r) => r.items),
  createPreset: (b: { name: string; settings: Settings }) => request<{ preset: Preset }>("/presets", "POST",
b).then((r) => r.preset),
  deletePreset: (id: string) => request<void>(`/presets/${id}`, "DELETE"),

  faqs: () => request<{ items: Faq[] }>("/faqs").then((r) => r.items),
};

export const errorText = (e: unknown) =>
  e instanceof ApiError ? [e.message, ...(e.details?.map((d) => `${d.field}: ${d.message}`) ?? [])].join(" — ") :
"Something went wrong.";
```

Listing 3: client/src/context/AuthContext.tsx

```
import { createContext, useContext, useEffect, useState, type ReactNode } from "react";
import { api, getToken, setToken, type User } from "@/lib/api";

interface AuthValue {
  user: User | null;
  loading: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (name: string, email: string, password: string) => Promise<void>;
  /** Used after social sign-in: store the token the server gave us and load the user. */
  loginWithToken: (token: string) => Promise<void>;
  logout: () => void;
}
const Ctx = createContext<AuthValue>(null!);
export const useAuth = () => useContext(Ctx);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(!!getToken());

  useEffect(() => {
    if (!getToken()) return;
    api.me().then(setUser).catch(() => setToken(null)).finally(() => setLoading(false));
  }, []);

  const value: AuthValue = {
    user, loading,
    login: async (email, password) => { const r = await api.login({ email, password }); setToken(r.token);
setUser(r.user); },
    register: async (name, email, password) => { const r = await api.register({ name, email, password });
setToken(r.token); setUser(r.user); },
    loginWithToken: async (token) => { setToken(token); try { setUser(await api.me()); } catch (e) { setToken(null);
throw e; } },
    logout: () => { setToken(null); setUser(null); },
  };
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
}
```

Listing 4: client/src/components/ascii/SavePanel.tsx

```
import { useState } from "react";
import { Save } from "lucide-react";
import { useAuth } from "@/context/AuthContext";
import { api, errorText, type NewArtwork } from "@/lib/api";

export type Snapshot = Omit<NewArtwork, "title" | "isPublic" | "tags">;

export default function SavePanel({ getSnapshot, onNeedAuth }: { getSnapshot: () => Snapshot | null; onNeedAuth: ()
=> void }) {
  const { user } = useAuth();
  const [title, setTitle] = useState("");
  const [isPublic, setIsPublic] = useState(false);
  const [tags, setTags] = useState("");
  const [msg, setMsg] = useState("");

  const save = async () => {
    const snap = getSnapshot();
    if (!snap) return setMsg("Create some ASCII art first.");
    if (!title.trim()) return setMsg("Give your art a title.");
    try {
      const tagList = [...new Set(tags.split(",").map((t) => t.trim().toLowerCase()).filter(Boolean))].slice(0, 5);
      await api.createArtwork({ ...snap, title: title.trim(), isPublic, tags: tagList });
      setMsg(isPublic ? "Saved — it is now in the public gallery." : "Saved to your gallery (private).");
setTitle(""); setTags("");
    } catch (e) { setMsg(errorText(e)); }
  };

  return (
    <div className="flex flex-col gap-2 border-t border-zinc-800 pt-4">
      <h3 className="text-sm font-semibold">Save to gallery</h3>
      {!user ? (
        <button onClick={onNeedAuth} className="rounded-lg border border-zinc-700 bg-zinc-900 px-3 py-2 text-sm
hover:border-emerald-400">Log in to save</button>
      ) : (<>
        <input value={title} onChange={(e) => setTitle(e.target.value)} maxLength={80} placeholder="Title"
className="rounded-lg border border-zinc-700 bg-zinc-900 px-3 py-2 text-sm outline-none focus:border-emerald-400" />
        <input value={tags} onChange={(e) => setTags(e.target.value)} placeholder="Tags, comma separated (max 5)"
className="rounded-lg border border-zinc-700 bg-zinc-900 px-3 py-2 text-sm outline-none focus:border-emerald-400" />
        <label className="flex items-center gap-2 text-sm text-zinc-300"><input type="checkbox" checked={isPublic}
onChange={(e) => setIsPublic(e.target.checked)} className="accent-emerald-400" /> Public</label>
        <button onClick={save} className="flex items-center justify-center gap-2 rounded-lg bg-emerald-400 px-3 py-2
text-sm font-semibold text-black hover:bg-emerald-300"><Save className="h-4 w-4" /> Save</button>
      </>)}
      {msg && <p className="text-xs text-amber-400">{msg}</p>}
    </div>
  );
}
```

Listing 5: server/src/app.js

```
import express from "express";
import cors from "cors";
import helmet from "helmet";
import rateLimit from "express-rate-limit";
import mongoose from "mongoose";
import { env } from "./config/env.js";
import authRoutes from "./routes/auth.js";
import artworkRoutes from "./routes/artworks.js";
import presetRoutes from "./routes/presets.js";
import faqRoutes from "./routes/faqs.js";
import { notFound, errorHandler } from "./middleware/error.js";

export const app = express();

app.use(helmet());
app.use(cors({ origin: env.clientOrigin }));
app.use(express.json({ limit: "6mb" }));

app.get("/api/health", (_req, res) =>
  res.json({ status: "ok", db: mongoose.connection.readyState === 1 ? "connected" : "disconnected" })
);

const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, limit: 100, standardHeaders: true, legacyHeaders:
false });
app.use("/api/auth", authLimiter, authRoutes);
app.use("/api/artworks", artworkRoutes);
app.use("/api/presets", presetRoutes);
app.use("/api/faqs", faqRoutes);

app.use(notFound);
app.use(errorHandler);
```

Listing 6: server/src/models/User.js

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true, minlength: 2, maxlength: 50 },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    passwordHash: { type: String, select: false },          // absent for accounts created via Google/GitHub
    googleId: { type: String, index: true, sparse: true },
    githubId: { type: String, index: true, sparse: true },
    avatar: { type: String, default: "" },
    providers: { type: [String], default: ["password"] },   // how this account can sign in
    role: { type: String, enum: ["user", "admin"], default: "user" },
  },
  { timestamps: true }
);

userSchema.set("toJSON", {
  transform: (_doc, ret) => { delete ret.passwordHash; delete ret.__v; return ret; },
});

export const User = mongoose.model("User", userSchema);
```

Listing 7: server/src/models/Artwork.js

```
import mongoose from "mongoose";

const settingsSchema = new mongoose.Schema(
  {
    cols: { type: Number, min: 20, max: 500, required: true },
    contrast: { type: Number, min: 0.3, max: 3, required: true },
    brightness: { type: Number, min: -0.5, max: 0.5, required: true },
    charset: { type: String, enum: ["detailed", "classic", "blocks", "binary", "matrix"], required: true },
    mode: { type: String, enum: ["green", "amber", "color"], required: true },
    invert: { type: Boolean, default: false },
    mirror: { type: Boolean, default: false },
  },
  { _id: false }
);

const artworkSchema = new mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true, index: true },
    title: { type: String, required: true, trim: true, maxlength: 80 },
    text: { type: String, required: true },            // the ASCII art itself (rows joined by \n)
    thumbnail: { type: String, default: "" },           // small data-URL preview for the gallery
    cols: { type: Number, required: true },
    rowCount: { type: Number, required: true },
    settings: { type: settingsSchema, required: true },
    sourceType: { type: String, enum: ["camera", "video", "image"], required: true },
    isPublic: { type: Boolean, default: false },
    tags: { type: [String], default: [] },
```

```
    likes: { type: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }], default: [] }, // who liked it (one like
per user)
    likeCount: { type: Number, default: 0, min: 0 },                                        // denormalised so "Top"
sorting is a plain index lookup
  },
  { timestamps: true }
);

artworkSchema.index({ isPublic: 1, createdAt: -1 });
artworkSchema.index({ isPublic: 1, likeCount: -1, createdAt: -1 });
artworkSchema.index({ isPublic: 1, tags: 1 });
artworkSchema.index({ user: 1, createdAt: -1 });

export const Artwork = mongoose.model("Artwork", artworkSchema);
```

Listing 8: server/src/middleware/auth.js

```
import jwt from "jsonwebtoken";
import { env } from "../config/env.js";
import { User } from "../models/User.js";
import { AppError, asyncHandler } from "./error.js";

export const signToken = (userId) => jwt.sign({ sub: String(userId) }, env.jwtSecret, { expiresIn:
env.jwtExpires });

async function loadUser(req) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) return null;
  const payload = jwt.verify(token, env.jwtSecret);
  const user = await User.findById(payload.sub);
  if (!user) throw new AppError(401, "User no longer exists");
  return user;
}

export const requireAuth = asyncHandler(async (req, _res, next) => {
  const user = await loadUser(req);
  if (!user) throw new AppError(401, "Authentication required");
  req.user = user;
  next();
});

export const optionalAuth = asyncHandler(async (req, _res, next) => {
  req.user = await loadUser(req);
  next();
});
```

Listing 9: server/src/middleware/error.js

```
import { ZodError } from "zod";

export class AppError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

export const notFound = (req, _res, next) => next(new AppError(404, `Route not found: ${req.method} $
{req.originalUrl}`));

// eslint-disable-next-line no-unused-vars
export function errorHandler(err, _req, res, _next) {
  let status = err.status || 500;
  let message = err.message;
  let details = err.details;

  if (err instanceof ZodError) {
    status = 400; message = "Validation failed";
    details = err.issues.map((i) => ({ field: i.path.join("."), message: i.message }));
  } else if (err.name === "CastError") { status = 400; message = "Invalid id"; }
  else if (err.name === "ValidationError") { status = 400; message = "Validation failed"; }
  else if (err.code === 11000) { status = 409; message = "That value already exists"; }
  else if (err.name === "JsonWebTokenError" || err.name === "TokenExpiredError") { status = 401; message = "Invalid
or expired token"; }
  else if (err.type === "entity.too.large") { status = 413; message = "Payload too large"; }
  else if (err.type === "entity.parse.failed") { status = 400; message = "Malformed JSON"; }

  if (status >= 500) { console.error(err); message = "Internal server error"; details = undefined; }
  res.status(status).json({ error: { message, details } });
}
```

Listing 10: server/src/validators.js

```
import { z } from "zod";
```

```
export const settingsSchema = z.object({
  cols: z.number().int().min(20).max(500),
  contrast: z.number().min(0.3).max(3),
  brightness: z.number().min(-0.5).max(0.5),
  charset: z.enum(["detailed", "classic", "blocks", "binary", "matrix"]),
  mode: z.enum(["green", "amber", "color"]),
  invert: z.boolean(),
  mirror: z.boolean(),
});

export const registerSchema = z.object({
  name: z.string().trim().min(2, "Name must be at least 2 characters").max(50),
  email: z.string().trim().toLowerCase().email("Enter a valid email"),
  password: z.string().min(8, "Password must be at least 8 characters").max(72),
});

export const loginSchema = z.object({
  email: z.string().trim().toLowerCase().email("Enter a valid email"),
  password: z.string().min(1, "Password is required"),
});

const tags = z.array(z.string().trim().toLowerCase().min(1).max(20)).max(5);

export const artworkCreateSchema = z.object({
  title: z.string().trim().min(1, "Title is required").max(80),
  text: z.string().min(1).max(300000),
  thumbnail: z.string().max(300000).startsWith("data:image/").optional(),
  cols: z.number().int().min(1).max(500),
  rowCount: z.number().int().min(1).max(2000),
  settings: settingsSchema,
  sourceType: z.enum(["camera", "video", "image"]),
  isPublic: z.boolean().default(false),
  tags: tags.default([]),
});

export const artworkUpdateSchema = z
  .object({ title: z.string().trim().min(1).max(80), isPublic: z.boolean(), tags })
  .partial()
  .refine((v) => Object.keys(v).length > 0, "Send at least one field to update");

export const artworkListQuery = z.object({
  scope: z.enum(["public", "mine", "liked"]).default("public"),
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(50).default(12),
  search: z.string().trim().max(50).optional(),
  sort: z.enum(["new", "top"]).default("new"),
  source: z.enum(["camera", "video", "image"]).optional(),
  tag: z.string().trim().toLowerCase().max(20).optional(),
});

export const presetSchema = z.object({
  name: z.string().trim().min(1, "Name is required").max(40),
  settings: settingsSchema,
});
```

Listing 11: server/src/controllers/auth.js

```
import bcrypt from "bcryptjs";
import { User } from "../models/User.js";
import { AppError, asyncHandler } from "../middleware/error.js";
import { signToken } from "../middleware/auth.js";

export const register = asyncHandler(async (req, res) => {
  const { name, email, password } = req.body;
  if (await User.exists({ email })) throw new AppError(409, "An account with that email already exists");
  const user = await User.create({ name, email, passwordHash: await bcrypt.hash(password, 10), providers:
["password"] });
  res.status(201).json({ token: signToken(user._id), user });
});

export const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email }).select("+passwordHash");
  if (user && !user.passwordHash) throw new AppError(401, `This account uses ${user.providers.join(" / ")} sign-in.
Use that button instead.`);
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) throw new AppError(401, "Incorrect email or
password");
  res.json({ token: signToken(user._id), user });
});

export const me = (req, res) => res.json({ user: req.user });
```

Listing 12: server/src/controllers/artworks.js

```
import { Artwork } from "../models/Artwork.js";
import { AppError, asyncHandler } from "../middleware/error.js";

const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
```

```
// Turn a stored document into what the client needs: no raw `likes` array, but a likedByMe flag.
function present(doc, user) {
  const { likes = [], ...rest } = doc;
  return { ...rest, likedByMe: !!user && likes.some((id) => String(id) === String(user._id)) };
}

// READ (list) — public gallery, or the signed-in user's own art. The heavy `text` field is left out.
export const list = asyncHandler(async (req, res) => {
  const { scope, page, limit, search, sort, source, tag } = req.query;
  const filter = {};
  if (scope === "mine") {
    if (!req.user) throw new AppError(401, "Authentication required");
    filter.user = req.user._id;
  } else if (scope === "liked") {
    if (!req.user) throw new AppError(401, "Authentication required");
    filter.isPublic = true;
    filter.likes = req.user._id;
  } else filter.isPublic = true;
  if (search) filter.title = new RegExp(escapeRegex(search), "i");
  if (source) filter.sourceType = source;
  if (tag) filter.tags = tag;
  const order = sort === "top" ? { likeCount: -1, createdAt: -1 } : { createdAt: -1 };

  const [items, total] = await Promise.all([
    Artwork.find(filter).select("-text").populate("user", "name").sort(order).skip((page - 1) *
limit).limit(limit).lean(),
    Artwork.countDocuments(filter),
  ]);
  res.json({ items: items.map((a) => present(a, req.user)), page, limit, total, pages: Math.ceil(total / limit) });
});

// READ (one) — includes the full ASCII text. Private art is only visible to its owner.
export const getOne = asyncHandler(async (req, res) => {
  const art = await Artwork.findById(req.params.id).populate("user", "name").lean();
  const isOwner = art && req.user && String(art.user._id) === String(req.user._id);
  if (!art || (!art.isPublic && !isOwner)) throw new AppError(404, "Artwork not found");
  res.json({ artwork: present(art, req.user) });
});

// CREATE
export const create = asyncHandler(async (req, res) => {
  const art = await Artwork.create({ ...req.body, user: req.user._id });
  const { text, likes, ...rest } = art.toObject();
  res.status(201).json({ artwork: { ...rest, likedByMe: false } });
});

// UPDATE (title / visibility / tags) — owner only
export const update = asyncHandler(async (req, res) => {
  const art = await Artwork.findOneAndUpdate({ _id: req.params.id, user: req.user._id }, req.body, { new: true,
runValidators: true }).select("-text").lean();
  if (!art) throw new AppError(404, "Artwork not found");
  res.json({ artwork: present(art, req.user) });
});

// DELETE — owner only
export const remove = asyncHandler(async (req, res) => {
  const art = await Artwork.findOneAndDelete({ _id: req.params.id, user: req.user._id });
  if (!art) throw new AppError(404, "Artwork not found");
  res.status(204).end();
});

// LIKE / UNLIKE (toggle) — any signed-in user, public art only (or your own).
// Each update is guarded by the filter, so a double-click can never count twice.
export const toggleLike = asyncHandler(async (req, res) => {
  const uid = req.user._id;
  const art = await Artwork.findById(req.params.id).select("isPublic user likes likeCount").lean();
  const isOwner = art && String(art.user) === String(uid);
  if (!art || (!art.isPublic && !isOwner)) throw new AppError(404, "Artwork not found");

  const liked = art.likes.some((id) => String(id) === String(uid));
  const updated = liked
    ? await Artwork.findOneAndUpdate({ _id: art._id, likes: uid }, { $pull: { likes: uid }, $inc: { likeCount:
-1 } }, { new: true }).select("likeCount").lean()
    : await Artwork.findOneAndUpdate({ _id: art._id, likes: { $ne: uid } }, { $addToSet: { likes: uid }, $inc:
{ likeCount: 1 } }, { new: true }).select("likeCount").lean();
  const likeCount = updated ? updated.likeCount : art.likeCount; // null = someone else's request already did it
  res.json({ liked: updated ? !liked : liked, likeCount });
});
```

Listing 13: server/src/routes/artworks.js

```
import { Router } from "express";
import * as c from "../controllers/artworks.js";
import { requireAuth, optionalAuth } from "../middleware/auth.js";
import { validate } from "../middleware/validate.js";
import { artworkCreateSchema, artworkUpdateSchema, artworkListQuery } from "../validators.js";

const r = Router();
r.get("/", optionalAuth, validate(artworkListQuery, "query"), c.list);
r.get("/:id", optionalAuth, c.getOne);
r.post("/", requireAuth, validate(artworkCreateSchema), c.create);
r.patch("/:id", requireAuth, validate(artworkUpdateSchema), c.update);
```

```
r.delete("/:id", requireAuth, c.remove);
r.post("/:id/like", requireAuth, c.toggleLike);
export default r;
```

Listing 14: server/tests/api.test.js

```
// Runs without a database: checks routing, validation and auth guards.
import test from "node:test";
import assert from "node:assert/strict";
import request from "supertest";
import { app } from "../src/app.js";

test("GET /api/health returns ok", async () => {
  const res = await request(app).get("/api/health");
  assert.equal(res.status, 200);
  assert.equal(res.body.status, "ok");
});

test("unknown route returns a JSON 404", async () => {
  const res = await request(app).get("/api/nope");
  assert.equal(res.status, 404);
  assert.ok(res.body.error.message);
});

test("register rejects a short password and bad email with 400", async () => {
  const res = await request(app).post("/api/auth/register").send({ name: "A", email: "nope", password: "123" });
  assert.equal(res.status, 400);
  assert.ok(res.body.error.details.length >= 2);
});

test("malformed JSON returns 400", async () => {
  const res = await request(app).post("/api/auth/login").set("Content-Type", "application/json").send("{bad");
  assert.equal(res.status, 400);
});

test("protected routes require a token", async () => {
  for (const [method, url] of [["get", "/api/auth/me"], ["post", "/api/artworks"], ["get", "/api/presets"],
["delete", "/api/artworks/123"], ["post", "/api/artworks/123/like"]]) {
    const res = await request(app)[method](url).send({});
    assert.equal(res.status, 401, `${method} ${url}`);
  }
});

test("a bad token is rejected with 401", async () => {
  const res = await request(app).get("/api/auth/me").set("Authorization", "Bearer not-a-real-token");
  assert.equal(res.status, 401);
});
```

SUBMISSION PAGE OF THE PAPER

Insert the submission page / acknowledgement of the paper here, if applicable.

PLAGIARISM REPORT

Insert the plagiarism (similarity) report here.